

None

Recuperación de la clave privada ECDSA
mediante reducción de retículos

47CON CTF 2026
Categoría: Cryptography

PABLO ALONSO CARRILLO

Abril 2026

Índice general

1	Descripción del reto	2
2	Reconocimiento y análisis inicial	2
2.1	Contenido del fichero	2
2.2	Identificación de la función hash	2
2.3	Descartes previos	2
3	Formalización y búsqueda del modelo	3
3.1	Del ECDSA al Hidden Number Problem	3
3.2	Hipótesis consideradas	3
3.3	Por qué LLL no es suficiente	4
4	Construcción del retículo y reducción	5
4.1	Transformación de las muestras	5
4.2	Estructura de la base	5
4.3	Extracción de la clave	5
4.4	Ejecución	6
5	Verificación y obtención de la flag	6
6	Análisis defensivo y mitigación	7
7	Conclusiones	7
A	Exploit (<code>solve.sage</code>)	8

1 Descripción del reto

El reto *None* plantea la recuperación de la clave privada de un esquema ECDSA sobre la curva P-256, a partir de 76 firmas acompañadas de un campo auxiliar que filtra información parcial sobre el *nonce* de cada una de ellas. No hay enunciado: el único material disponible es un fichero JSON y el propio nombre del reto, que ya constituye la primera pista, dado que *none* es *nonce* sin la letra *c*.

La resolución pasa por formalizar el problema como un *Hidden Number Problem* y resolverlo mediante reducción de retículos. La parte más compleja del reto no es la construcción del retículo, que es un procedimiento estándar, sino determinar qué información codifica el campo auxiliar y comprobar que LLL no basta, siendo necesario emplear BKZ con bloques de tamaño considerable.

2 Reconocimiento y análisis inicial

2.1 Contenido del fichero

El fichero `challenge.json` contiene la identificación de la curva (P-256), la clave pública Q , que he comprobado previamente que está sobre la curva, y un total de 76 firmas correspondientes a los mensajes `message-0` a `message-75`, con el campo `m` codificado en hexadecimal sobre la representación ASCII del mensaje.

El elemento que singulariza el reto es un campo `hint` por firma, con valores enteros entre 0 y 15. La presencia de un campo que filtra información asociada a cada firma es un indicio claro de que el problema gira en torno a *nonces* sesgados.

2.2 Identificación de la función hash

Antes de formalizar el ataque es fundamental conocer qué función resumen se ha empleado, ya que el valor h entra directamente en las ecuaciones. Esto puede determinarse sin conocer la clave privada, dado que la verificación de una firma ECDSA únicamente requiere la clave pública:

Código 1: Verificación de firma para identificar la función hash.

```
1 def verify(h, r, s):
2     w = pow(s, -1, n)
3     P = (h * w % n) * G + (r * w % n) * Qpub
4     return P[0] % n == r
```

Tras probar la primera firma con SHA-256, SHA-1 y el mensaje interpretado como entero, únicamente SHA-256 verifica correctamente. El esquema emplea por tanto SHA-256 sobre los bytes del mensaje sin truncar, lo cual resulta coherente porque P-256 y SHA-256 comparten el mismo tamaño de salida. Comprobar el hash en esta fase evita arrastrar un error silencioso: si se hubiera asumido SHA-1 o un truncamiento, todos los coeficientes del retículo habrían sido incorrectos y el ataque habría fallado sin causa aparente.

2.3 Descartes previos

Antes de construir el retículo he descartado algunas de las vías de ataque más sencillas:

- **Reutilización de *nonce*.** No existe ningún valor de r ni de s repetido, por lo que el común ataque de dos firmas con el mismo *nonce* queda descartado.
- ***hint* derivado de datos públicos.** He comprobado si *hint* coincidía con el nibble alto o bajo, la longitud en bits, o el módulo 16 y 17 de r , s , h , s^{-1} , $s^{-1}r$, $s^{-1}h$, rs y $r + s$. Ninguna combinación destaca sobre el azar, así que *hint* no depende de los datos públicos de la firma. Eso deja al *nonce* como única fuente posible de información.

- **Distribución de `hint`.** Los valores se reparten de forma uniforme entre las 16 categorías posibles: un test χ^2 da 14.9 con 15 grados de libertad, justo lo esperable si no hay sesgo. Esto descarta de entrada las interpretaciones que implicarían un sesgo estructural, como contar los bits cero a la izquierda del *nonce*, que producirían una distribución geométrica con el valor 0 apareciendo la mitad de las veces.

3 Formalización y búsqueda del modelo

3.1 Del ECDSA al Hidden Number Problem

La ecuación de firma ECDSA es $s = k^{-1}(h + r \cdot d) \bmod n$, donde k es el *nonce* (un valor aleatorio distinto para cada firma) y d es la clave privada. Despejando el *nonce*:

$$k \equiv t \cdot d + u \pmod{n}, \quad \text{con } t = s^{-1}r \text{ y } u = s^{-1}h.$$

Los coeficientes t y u se calculan a partir de la firma y del mensaje, de modo que cada firma relaciona linealmente dos incógnitas. La seguridad de ECDSA depende de que k sea aleatorio, uniforme y secreto. Si se cumplen esas condiciones, conocer r y s de muchas firmas no aporta información sobre d : cada firma introduce una incógnita nueva y el sistema tiene infinitas soluciones.

El problema aparece cuando algo de k se filtra. En cuanto se conoce parte del *nonce*, cada firma deja de ser una ecuación ambigua y pasa a ser una desigualdad sobre d : el valor $k_i = t_i d + u_i \bmod n$ queda restringido a un subconjunto pequeño de $\mathbb{Z}_n$. Con suficientes desigualdades independientes, la clave privada se puede aislar y recuperar mediante reducción de retículos. En este reto, el campo *hint* es precisamente esa filtración: 4 bits por firma, 76 firmas, 304 bits de información frente a los 256 bits de n .

Este es el planteamiento del *Hidden Number Problem* de Boneh y Venkatesan, aplicado a DSA por Howgrave-Graham y Smart y por Nguyen y Shparlinski. El problema se reduce a encontrar el d que satisface

$$0 \leq (t_i \cdot d + u_i \bmod n) < B_i$$

para todas las muestras, donde B_i es la cota superior de la parte desconocida del *nonce*. El ataque es viable precisamente porque $B_i < n$: cuanto más pequeño sea el rango en el que vive el *nonce*, más margen tiene el retículo para aislar d . El trabajo principal del reto consiste en determinar qué modelo de B_i y qué corrección sobre u_i codifica el campo *hint*.

3.2 Hipótesis consideradas

Un campo de 4 bits admite al menos siete lecturas razonables. Cuatro de ellas implican una distribución sesgada del *hint* y caen por la observación de la sección 2.3. Las tres restantes son compatibles con un nibble uniforme y requieren ser probadas con el ataque completo, empleando el mismo retículo y modificando únicamente la terna (t_i, u_i, B_i) .

Hipótesis	Modelo de la muestra	Resultado
Bits cero a la izquierda	$k < n \gg \text{hint}$	Descartada por χ^2
Ídem, cota $2^{255-\text{hint}}$	$k < 2^{255-\text{hint}}$	Descartada por χ^2
Ceros a la derecha	$k = e \cdot 2^{\text{hint}}$	Descartada por χ^2
Longitud en bits	$k < 2^{240+\text{hint}}$ y variantes	Descartada por χ^2
Nibble alto	$k = \text{hint} \cdot 2^{252} + e, e < 2^{252}$	Falla hasta BKZ-70
Nibble en posición j	$k = \text{hint} \cdot 2^j + e, \text{ barrido de } j$	Falla hasta BKZ-65
Nibble bajo	$k = 16e + \text{hint}, e < n/16$	Resuelto en BKZ-60

Cuadro 1: Interpretaciones consideradas del campo `hint`.

La interpretación correcta resulta ser la más simple de todas: $\text{hint} = k \bmod 16$, es decir, el nibble bajo del *nonce*. Indicar hasta qué bloque se llevó cada hipótesis fallida no es un detalle menor, por la razón que se explica a continuación.

3.3 Por qué LLL no es suficiente

Esta sección recoge la principal dificultad del reto. Con 4 bits por firma y 76 firmas se dispone de 304 bits de fuga frente a los 256 bits de n , de modo que la información, en principio, sobra. Sin embargo, al calcular los parámetros del retículo concreto, de dimensión 78, el margen resulta mucho más estrecho de lo que cabría esperar:

$$\log_2(\det M) = 25008, \quad \text{GH} = \det \cdot \sqrt{\frac{78}{2\pi e}} \approx 2^{321.71}, \quad \|\mathbf{v}\| \approx 2^{321.48}.$$

El vector objetivo es solo un 15 % más corto que el mínimo que predice la heurística gaussiana para un retículo aleatorio de ese tamaño y determinante. Un cociente de 0.85 deja muy poco margen, y eso implica que es necesario emplear BKZ con bloques grandes: LLL devuelve un primer vector por encima del objetivo, y con bloques de 20 o 30 tampoco se alcanza.

De aquí se extrae la lección más útil del reto: que una reducción falle no refuta la hipótesis. Solo indica que ese ataque, con esa potencia, no ha llegado lo suficientemente lejos. Conviene separar la idea de que el modelo sea incorrecto de que la reducción sea insuficiente, porque confundir ambos casos lleva a descartar modelos que en realidad son correctos.

Para no caer en ese error he calibrado la potencia de la herramienta con datos sintéticos: generando firmas propias sobre P-256 con el mismo sesgo y una clave conocida, y midiendo qué bloque es necesario para romperlas.

Código 2: Generación de instancias sintéticas para calibración.

```

1 random.seed(7)
2 d0 = random.randrange(1, n)
3 for i in range(76):
4     hint = random.randrange(16)
5     k = 16 * random.randrange(n // 16) + hint
6     R = k * G; r = R[0] % n
7     h = int.from_bytes(hashlib.sha256(f'm{i}'.encode()).digest(), 'big') % n
8     s = pow(k, -1, n) * (h + r * d0) % n

```

Sobre varias instancias sintéticas, la clave se recupera en bloques de entre 58 y 65, en menos de un minuto cada una, y ninguna cae con LLL ni con bloques pequeños. Esto proporciona el umbral de potencia necesario y permite interpretar los resultados negativos: una hipótesis llevada por encima de ese umbral sí que podría descartarse con bastante confianza.

4 Construcción del retículo y reducción

4.1 Transformación de las muestras

Con el modelo $k = 16e + \text{hint}$, la incógnita pasa a ser $e = (k - \text{hint})/16$, acotada por $n/16$. Multiplicando por el inverso de 16 módulo n se obtiene una muestra HNP en forma canónica:

Código 3: Preparación de las muestras según el modelo del nibble bajo.

```

1 # k = t*d + u (mod n), con k = 16*e + hint -> e = (t*d + u - hint)/16
2 inv16 = int(pow(16, -1, n))
3 B = n // 16 + 1
4
5 raw, T, U = [], [], []
6 for sg in sigs:
7     h = int.from_bytes(hashlib.sha256(bytes.fromhex(sg['m'])).digest(), 'big') % n
8     si = int(pow(int(sg['s']), -1, n))
9     t, u = si * int(sg['r']) % n, si * h % n
10    raw.append((t, u, int(sg['hint'])))
11    T.append(t * inv16 % n)
12    U.append((u - int(sg['hint'])) * inv16 % n)

```

4.2 Estructura de la base

Con m muestras se construye una base de dimensión $m + 2 = 78$. Se emplea un factor de escala $c = W/B$, con $W = 2^{320}$, para igualar la magnitud de todas las coordenadas del vector objetivo; sin él, el retículo queda desequilibrado y las coordenadas asociadas a d dominan sobre las de los e_i .

$$M = \begin{pmatrix} nc & 0 & \cdots & 0 & 0 & 0 \\ 0 & nc & \cdots & 0 & 0 & 0 \\ \vdots & & \ddots & & \vdots & \vdots \\ T_0c & T_1c & \cdots & T_{m-1}c & W/n & 0 \\ u'_0c & u'_1c & \cdots & u'_{m-1}c & 0 & W \end{pmatrix}$$

Las m primeras filas, diagonales y múltiplos de n , permiten la reducción modular. La fila m contiene los coeficientes de d y la fila $m + 1$ los términos independientes, aplicados de forma centrada, $u'_i = u_i - B/2$, de manera que en lugar de buscar e_i en $[0, B)$ se busca e'_i en $[-B/2, B/2)$. El centrado equivale a ganar un bit de fuga por muestra. Con un margen tan justo, ese bit cuenta.

La combinación $d \cdot (\text{fila } m) + 1 \cdot (\text{fila } m+1)$, reducida módulo n mediante las filas diagonales, produce el vector

$$\mathbf{v} = (e'_0c, \dots, e'_{m-1}c, d \cdot W/n, W),$$

con todas las coordenadas del orden de W , frente a los vectores típicos del retículo, mucho mayores.

4.3 Extracción de la clave

Un detalle que ahorra bastante tiempo de depuración es que no resulta fiable buscar d en la coordenada m de la primera fila. El vector que encuentra el algoritmo puede ser un múltiplo del objetivo, su opuesto, o una combinación con signos distintos de los esperados; además, las primeras filas pueden estar ocupadas por vectores cortos del retículo que no contienen información sobre d . Resulta mucho más fiable recorrer todas las coordenadas de todas las filas, interpretar cada una como un candidato e_i y despejar.

Código 4: Extracción robusta de la clave privada.

```

1 def hunt(R):
2     # cualquier coordenada de cualquier fila puede ser un e_i valido
3     seen = set()
4     for row in R.rows():
5         for i in range(m):
6             for sign in (1, -1):
7                 e = sign * int(row[i]) // c + B // 2
8                 d = (e - U[i]) * inv[i] % n
9                 if d in seen or not 0 < d < n:
10                     continue
11                 seen.add(d)
12                 # filtro rapido antes de multiplicar en la curva
13                 if all(((tt * d + uu) % n) % 16 == hh for tt, uu, hh in raw[:4]):
14                     if d * G == Qpub:
15                         return d
16     return None

```

Verificar un candidato mediante multiplicación escalar sobre la curva tarda aproximadamente 7 ms, y cada ronda genera del orden de 12.000 candidatos, lo que supone unos 80 segundos por intento. Un filtro previo de coste despreciable consiste en comprobar que el candidato reproduce los `hint` de cuatro firmas. Con eso, solo los supervivientes llegan a tocar la curva, por lo que ese tiempo se reduce prácticamente a cero.

4.4 Ejecución

El ataque se ha ejecutado con SageMath 10.6, que incluye el BKZ de `fpLLL` y la aritmética de curva elíptica de serie. Es necesario pasar `proof=False`: por defecto Sage hereda `proof.linear_algebra()` y exige una reducción sin poda, que en dimensión 78 con bloque 60 tarda excesivo tiempo sin ninguna ganancia útil en este caso.

```
1 sage solve.sage challenge.json
```

```

(sage) prodigy@UbuntuCTF:~/Escritorio$ sage solve.sage challenge.json
[+] BKZ-30 -> None
[+] BKZ-45 -> None
[+] BKZ-55 -> None
[+] BKZ-60 -> 3512966322301963193522536730116398635766819581017145228026797213710394932195
d = 3512966322301963193522536730116398635766819581017145228026797213710394932195
FLAG: 47CON{7c444bae5fdae75274cbe577f60dcfba8b07ca94f620dbc8520ddc371d13be3}

```

Figura 1: Salida de `solve.sage`. Los bloques 30, 45 y 55 no localizan el vector objetivo; el bloque 60 recupera la clave privada.

5 Verificación y obtención de la flag

La clave recuperada pasa dos comprobaciones. La primera es que $d \cdot G = Q_{\text{pub}}$, lo que confirma que se trata de la clave privada asociada a la pública del fichero. La segunda reconstruye el *nonce* de las 76 firmas con $k_i = t_i d + u_i \bmod n$ y comprueba que $k_i \bmod 16$ coincide con el `hint` en los 76 casos, lo que valida el modelo.

Conocida la clave, se pueden cerrar además las hipótesis que habían quedado en duda, porque ya no dependen de umbrales de reducción sino de una evaluación directa. El nibble alto acierta en solo 5 de 76, cifra compatible con el azar (cabría esperar $76/16 = 4.75$); y las longitudes de los k_i se reparten entre 250 y 256 bits, por lo que ningún *nonce* está truncado. Las dos hipótesis que fallaron en el retículo eran, efectivamente, falsas.

```

1 d = 3512966322301963193522536730116398635766819581017145228026797213710394932195
2 hex = 7c444bae5fdae75274cbe577f60dcfba8b07ca94f620dbc8520ddc371d13be3

```

```
47CON{7c444bae5fdae75274cbe577f60dcfba8b07ca94f620dbc8520ddc371d13be3}
```

La clave tiene 251 bits, y por eso el hexadecimal ocupa 63 caracteres en lugar de 64: `format(d, 'x')` no rellena con ceros a la izquierda. La flag se aceptó en ese formato.

6 Análisis defensivo y mitigación

La vulnerabilidad no se encuentra en la curva ni en el algoritmo de firma, que son sólidos, sino en la generación del *nonce*. El resultado merece ser destacado: han bastado cuatro bits filtrados por firma, y además los cuatro *menos* significativos, que en principio podrían parecer los menos comprometedores, para recuperar por completo una clave de 256 bits con menos de cien firmas.

- **Generación determinista del *nonce*.** RFC 6979 deriva k de forma determinista a partir de la clave privada y del mensaje mediante HMAC, lo que elimina la dependencia del generador de aleatorios y garantiza un *nonce* indistinguible de uniforme sin necesitar entropía en el momento de firmar.
- **Protección frente a canales laterales.** En sistemas reales la fuga no procede de un campo explícito como el de este reto, sino de diferencias de tiempo, accesos a caché o consumo durante la multiplicación escalar. Ataques como Minerva o TPM-Fail han demostrado que unos pocos bits bastan para recuperar la clave en producción. La mitigación pasa por aritmética en tiempo constante y por evitar ramificaciones que dependan del secreto.
- **Exposición mínima de metadatos.** Cualquier dato auxiliar publicado junto a una firma debe considerarse explotable. El campo `hint` de este reto es la versión explícita y didáctica de lo que en un sistema real sería un metadato aparentemente inofensivo.

7 Conclusiones

El reto deja claro que la seguridad de un esquema criptográfico no depende solo de que el problema matemático subyacente sea duro, sino de que cada pieza de la implementación esté correctamente construida. El logaritmo discreto sobre P-256 sigue intacto después de este ataque; lo que se ha roto es la hipótesis de que el *nonce* era uniforme, y con eso ha bastado.

En cuanto al método, extraigo tres conclusiones. La primera es que un campo sospechoso no siempre significa lo que parece: un nibble uniforme admite al menos siete lecturas razonables, y enumerarlas todas antes de ponerse a programar ahorra muchas vueltas. La segunda, y probablemente la más útil, es que el fallo de una reducción no refuta un modelo, por lo que conviene calibrar la potencia de la herramienta sobre datos sintéticos con clave conocida antes de descartar nada; de hecho, ya había probado con anterioridad la hipótesis correcta, pero la descarté por haberla atacado únicamente con LLL. La tercera es que comparar la norma objetivo con la heurística gaussiana es un cálculo de apenas treinta segundos que orienta todo el ataque, porque un cociente de 0.85 avisa desde el principio de que van a ser necesarios bloques grandes.

A Exploit (solve.sage)

El exploit completo se reproduce con:

```
1 sage solve.sage challenge.json
```

Los datos de entrada están en `challenge.json`, incluido en el repositorio. La ejecución completa tarda menos de dos minutos en un equipo estándar con SageMath 10.6 instalado.

[illegible]

```
51     for row in R.rows():
52         for i in range(m):
53             for sign in (1, -1):
54                 e = sign * int(row[i]) // c + B // 2
55                 d = (e - U[i]) * inv[i] % n
56                 if d in seen or not 0 < d < n:
57                     continue
58                 seen.add(d)
59                 # filtro rapido antes de multiplicar en la curva
60                 if all(((tt * d + uu) % n) % 16 == hh for tt, uu, hh in raw[:4]):
61                     if d * G == Qpub:
62                         return d
63     return None
64
65 R = M.LLL()
66 d = hunt(R)
67 for blk in (30, 45, 55, 60, 62, 65, 68, 70):
68     if d:
69         break
70     R = R.BKZ(block_size=blk, proof=False)
71     d = hunt(R)
72     print('[+] BKZ-%d -> %s' % (blk, d))
73
74 if d is None:
75     sys.exit('no recuperada: prueba a subir el tamaño de bloque')
76
77 print('d =', d)
78 print('FLAG: 47CON{%s}' % format(d, 'x'))
```